

Walmart Medallion Data Pipeline

MongoDB → PostgreSQL → dbt Bronze → Silver → Gold

Analytics-Engineering Technical Report

Version 0.2.2 | 2026-10-14 | 54c9cae

Nitin | MIT License

Document control

Item	Detail
Title	Walmart Medallion Data Pipeline — Technical Report
Version	0.2.2 (from `pyproject.toml`)
Date	2026-10-14
Commit	`54c9cae`
Author	Nitin
Status	Final
License	MIT (see `LICENSE`)

Abstract

A MongoDB e-commerce database is mirrored into PostgreSQL and curated through bronze (raw), silver (typed, nine models) and gold (star schema, seven dimensions plus one fact) layers. One fixed eight-stage pipeline with fail-fast gates runs identically on Windows, Docker, Airflow and Make. Quality is triple-checked per layer by dbt tests, a standalone SQL suite and Great Expectations, with per-table data contracts synced to the live database. A seven-page Streamlit dashboard serves gold; CI replays the pipeline against live Postgres. This report documents the architecture, stages, models, gates, app, operations and measured metrics of pipeline version 0.2.2.

How to read this report

Colour legend, notation and icons used throughout the report.

Layer colours never vary: bronze marks raw bronze content, silver cleansed silver, gold modelled gold. Status colours mark outcomes: green pass, amber warning, red failure. `monospace` names a file, path or identifier; `monospace` names a command to run. Numbered circles (0) name pipeline stages. Every chapter opens with a lead and an at-a-glance box and closes with key takeaways; captions state the takeaway first and the CSV or file source last.

Contents

Document control ii

Abstract iii

How to read this report iv

1 Executive summary 1
- 1.1 Problem and solution . . . 1
- 1.2 Headline numbers . . . 2

2 Architecture 3
- 2.1 End-to-end flow . . . 3
- 2.2 Layer responsibilities . . . 3
- 2.3 Infrastructure view . . . 5

3 Bronze and extraction 6
- 3.1 Source tables . . . 6
- 3.2 Watermark rule . . . 7
- 3.3 Upsert mechanics . . . 7
- 3.4 Control tables and CLI . . . 8

4 Silver 9
- 4.1 Model map . . . 9
- 4.2 Cleaning patterns . . . 10

5 Gold star schema 12
- 5.1 Schema . . . 12
- 5.2 Design decisions . . . 13

6 Pipeline orchestration 14
- 6.1 Stages . . . 14
- 6.2 Runner parity . . . 15
- 6.3 Airflow . . . 15

7 Data quality **18**
7.1 Defense in depth . . . 18
7.2 What each system uniquely catches . . . 19

8 Data contracts **20**
8.1 Anatomy . . . 20
8.2 Sync discipline . . . 21

9 Dashboard **22**
9.1 Layering . . . 22
9.2 Pages . . . 23

10 CI, CD and Docker **24**
10.1 CI . . . 24
10.2 Docker . . . 25

11 Operations and security **27**
11.1 Operate . . . 27
11.2 Secure . . . 27

12 Results and metrics **29**
12.1 Components and coverage by layer . . . 29
12.2 Row counts . . . 29
12.3 Code size and test composition . . . 29
12.4 Stage durations . . . 32
12.5 Dashboard coverage . . . 34

13 Limitations and roadmap **35**
13.1 Limitations . . . 35
13.2 Roadmap . . . 36

A File inventory **37**

B Command cheat-sheet **39**

C Environment variable reference **40**

D Glossary **41**

E Full entity-relationship diagram **42**

F Sample data contract **44**

G Change log **46**

List of Figures

2.1 System overview. Raw data flows left to right; quality gates sit between layers; Airflow schedules runs above. Takeaway: no path reaches gold without passing a gate. 4
2.2 What each layer holds and guarantees, with one example transformation each. Takeaway: cleaning happens once in silver, never in consumers. 4

3.1 Extractor flowchart from discovery to report. Takeaway: every path ends in validation, and control tables record what happened. 8

4.1 Silver dependencies. Lookup models distil bronze text columns; entity models merge and join them. Takeaway: FK resolution happens here, once. 9

5.1 Gold ERD. Arrows are FK reads at build time. Takeaway: every analytic path joins through the fact or `dim_orders`. 12

6.1 The eight stages with their tools. Banners print `STEPn/7` because stages are numbered 0–7. Takeaway: order never varies, whatever runs it. 15
6.2 Five runners, one stage list. Takeaway: pick by platform, not by behavior. . . 16
6.3 Pipeline DAG tasks and notification branch. Takeaway: linear gate chain, then one result sweep and a single email decision. 16

7.1 Which system checks what per layer. Overlap is deliberate redundancy across runners. Takeaway: each column catches something the others cannot. . . . 18

8.1 Contract lifecycle. The CI gate runs `--check-only` and blocks on drift. Takeaway: contracts stay truthful because the database rewrites them. 20

9.1 Dashboard data path with cache TTLs. Takeaway: one join graph, one SQL file, one theme. 22

10.1 CI job DAG and publish triggers. Takeaway: everything expensive waits on lint; integration replays the pipeline order against live Postgres. 25
10.2 Images and Compose services. Takeaway: one image runs the pipeline, the other schedules it, Compose wires them locally. 25

12.1 Components per layer: bronze has no dbt models (sources are declared, not built). Takeaway: silver carries the most machinery. Source: `reports/data/components_by_layer.csv`. 30
12.2 Defense-in-depth composition. Takeaway: no layer relies on a single system. Source: `reports/data/components_by_layer.csv`. 30
12.3 Promotion parity on a log axis; `employees` is skipped because it has no gold dimension by design. Takeaway: every promoted table arrives complete. Source: `reports/data/layer_rowcounts.csv` (contract machine fields). . 31
12.4 Bronze counts are one row per table: the seeded snapshot recorded in the contracts, shown to prove the pipeline, not production volume. Source: `reports/data/extract_rows.csv`. 31
12.5 Python, SQL, YAML and shell lines per area, excluding vendored and generated trees. Takeaway: ops scripts outweigh the runtime; the warehouse itself is compact. Source: `reports/data/loc_by_area.csv`. 32
12.6 Suite sizes as test-function and check counts. Takeaway: dbt carries two thirds of all checks. Source: `reports/data/tests_pyramid.csv`. 32
12.7 Exact counts behind Fig. 12.6: unit and smoke count defined test functions; dbt counts `tests:` entries in `schema.yml`. Source: `reports/data/tests_pyramid.csv`. 33
12.8 Extract and the dbt builds dominate; SQL gates take about a second. Source: newest `logs/pipeline_*.log` via the collector. 33

E.1 Gold ERD, repeated large for reference. The fact joins dims on surrogate and natural keys; products snowflake into brands and categories. Source: `dbt/models/gold/*.sql`. 43

List of Tables

1.1 Headline numbers; each is regenerated from the repo by the metrics collector. 2

2.1 Medallion layer contract. Source: `docs/architecture.md`. 4

3.1 Bronze mirrors one collection each. Source: `dbt/models/bronze/source.yml`. 6

4.1 The nine silver models. Source: `dbt/models/silver/*.sql`. 10

5.1 Gold models and grains. Source: `dbt/models/gold/*.sql`. 13

6.1 Stage commands. Exit codes: `sql_test.py` 0/1/2, GX 0/1/2, extract non-zero on any failed collection. 14

7.1 Check counts by layer and system. Sources: `dbt/models/*/schema.yml`, `tests/data-quality/`, `gx/suites/`. 19

9.1 Dashboard pages. Source: `dashboard/pages/*.py`. 23

10.1 CI jobs. Source: `.github/workflows/ci.yml`. 24

12.1 Pages, figure-builder calls and gold tables read. Every business page shares the `BASE_FROM` graph; the health page additionally reads bronze control tables. Source: `reports/data/dashboard_pages.csv`. 34

13.1 Known limits. Each has a roadmap item below. 35

B.1 Cheat-sheet. 39

D.1 Glossary. 41

G.1 Report change log. 46

1

Executive summary

A batch incremental warehouse with triple-checked gates, running one stage list on every platform.

At a glance

- Problem: operational MongoDB cannot serve typed, historical analytics directly.
- Solution: watermark extraction, dbt medallion layers, gates at every hand-off.
- Proof: 17 models, 214 checks, 25 contracts, 8 stages, 7 CI jobs, measured durations.

1.1 Problem and solution

Analytics needs typed history and dimensions; MongoDB stays write-optimized. The pipeline bridges them: PySpark extraction with watermark upserts lands raw bronze, nine dbt models clean it into silver, eight models star it into gold, and a Streamlit app serves seven pages from one shared join graph. Three independent quality systems (dbt tests, SQL suite, Great Expectations) gate each hand-off, and per-table contracts record the agreed shape against the live database.

1.2 Headline numbers

Measure	Count
dbt models (silver + gold)	17
Quality checks (dbt + SQL + GX)	214
Data contracts	25
Pipeline stages	8
Runners (ps1, bat, Docker, Airflow, Make)	5
CI jobs	7
Dashboard pages	7

Table 1.1: Headline numbers; each is regenerated from the repo by the metrics collector.

The stack is Python with uv, PySpark 3.5, PostgreSQL 16, dbt Core, Airflow 3.3, Great Expectations 1.x and Docker, all pinned in `uv.lock`. Figure 2.1 shows the hero flow; the chapters that follow prove each number in Table 1.1.

Key takeaways

- One warehouse, one stage list, three quality systems, five runners.
- Every number in this report traces to a repo file or generated CSV.
- Read Chapters 2–7 for the build, 9–11 for the surroundings, 12 for the measurements.

2

Architecture

One warehouse, three layers, five runners: the whole system on two pages.

At a glance

- MongoDB (operational) → PostgreSQL `walmart_db` (bronze → silver → gold) → dashboard and reports.
- Every layer hand-off is a quality gate; a failed gate stops the run.
- The same eight stages run on Windows, in Docker, on Airflow and from Make.

2.1 End-to-end flow

Figure 2.1 shows the complete path. `src/jobs/extract.py` copies each MongoDB collection into its own bronze table inside PostgreSQL 16. Two dbt selections promote the data: nine silver models clean and type it, eight gold models shape it into a star schema. Consumers (Streamlit dashboard, `sql/analytics`, `reports/`) read gold only.

2.2 Layer responsibilities

Figure 2.2 compresses the contract in Table 2.1 to one glance; the table states it precisely.

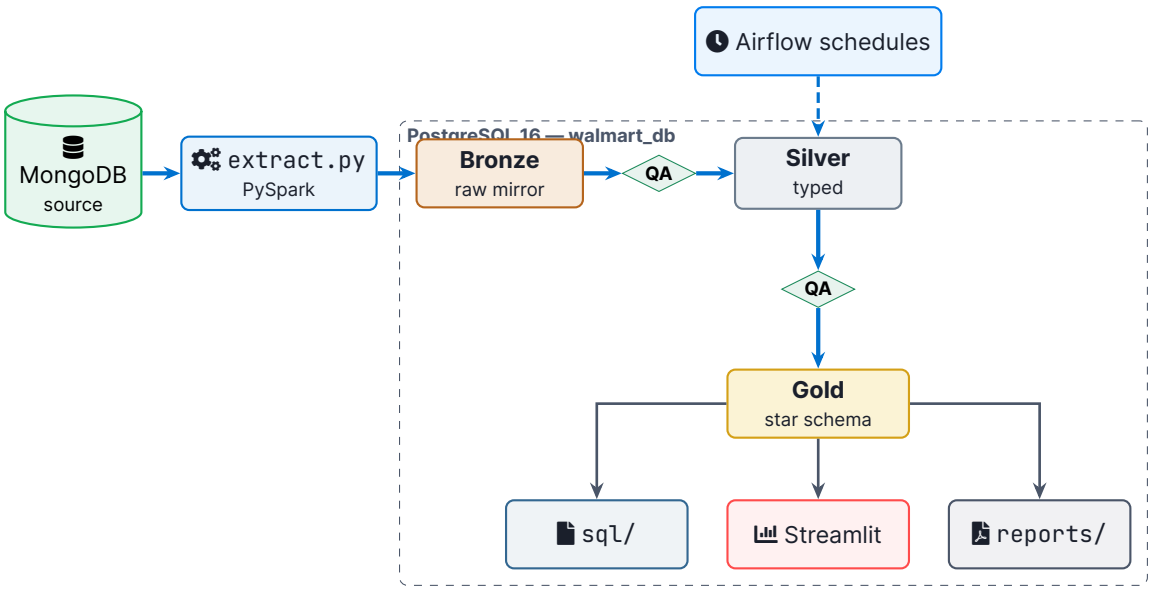


Figure 2.1: System overview. Raw data flows left to right; quality gates sit between layers; Airflow schedules runs above. Takeaway: no path reaches gold without passing a gate.

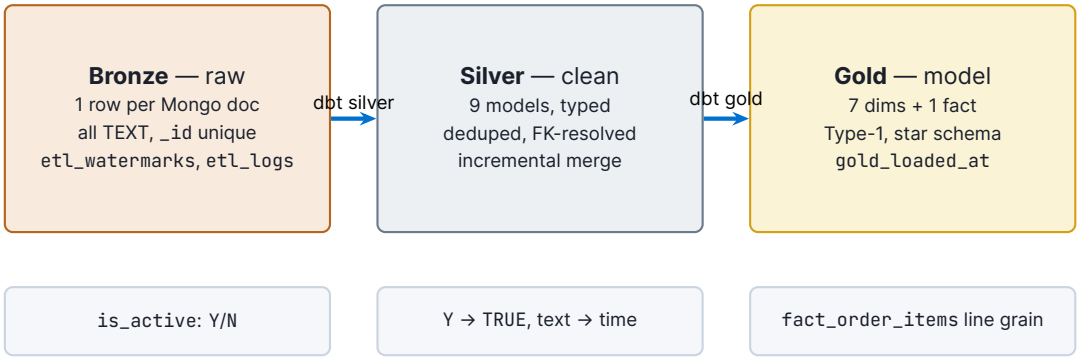


Figure 2.2: What each layer holds and guarantees, with one example transformation each. Takeaway: cleaning happens once in silver, never in consumers.

Layer	Holds	Guarantee
bronze	Raw 1:1 mirror of MongoDB, plus `etl_watermarks` and `etl_logs`	Source fidelity; `_id` unique
silver	Nine typed, trimmed, deduplicated models	One clean row per source document
gold	Seven dimensions plus `fact_order_items`	Analytics-ready star, Type-1

Table 2.1: Medallion layer contract. Source: `docs/architecture.md`.

2.3 Infrastructure view

Postgres is `walmart_db`; schema names default to `bronze`/`silver`/`gold` through `POSTGRES_SCHEMA_*` in `.env`, read once by `src/utils/engine.py`. All runners set `PYTHONPATH` to the project root so `src.*` resolves identically. The localhost rule (`docs/architecture.md`): `.env` keeps `localhost` for local `uv run`; only containers rewrite to `host.docker.internal` in the DAG preamble and entrypoint. `.env` itself is never edited.

Key takeaways

- Bronze mirrors, silver cleans, gold models; consumers read gold only.
- Gates between layers fail the run; nothing half-promoted ever lands downstream.
- One `.env`, validated once, shared by every runner and the app.

3

Bronze and extraction

Six MongoDB collections become six raw Postgres tables, incrementally and auditably.

At a glance

- `src/jobs/extract.py` is the only file that touches MongoDB.
- Collections are auto-discovered; the watermark column follows a fixed priority.
- First load appends; later loads upsert through a staging table keyed on `_id`.

3.1 Source tables

Table 3.1 lists the six mirrors; collection discovery below explains why the list never hard-codes them.

MongoDB collection	bronze table
`customers`	`bronze.customers`
`orders`	`bronze.orders`
`order_items`	`bronze.order_items`
`products`	`bronze.products`
`stores`	`bronze.stores`
`employees`	`bronze.employees`

Table 3.1: Bronze mirrors one collection each. Source: `dbt/models/bronze/source.yml`.

`discover_collections()` lists collection names and drops anything starting with `system.`; no list is ever hard-coded, and `--tables` filters for ad-hoc runs.

3.2 Watermark rule

Each collection is sampled once and the first present candidate wins:

$$\texttt{updated_timestamp} \succ \texttt{updated_at} \succ \texttt{created_timestamp} \succ \texttt{created_at}$$

`updated_*` outranks `created_*` because mutated documents (an order moving to `Cancelled`) must re-sync, not just new inserts. The constant lives in `src/jobs/extract.py` as `INCREMENTAL_COLUMN_CANDIDATES` (lines 151–156). Without any candidate the collection reloads fully with a warning.

The Spark read filters server-side with a string comparison, which is intentional: the source stores timestamps as `YYYY-MM-DDHH:MM:SS` strings, and MongoDB never matches across BSON types in `\$gt`.

src/jobs/extract.py (excerpt)

```
watermark_str = since.strftime("%Y-%m-%d %H:%M:%S")
pipeline = json.dumps(
    [{"$match": {incremental_column: {"$gt": watermark_str}}}]
)
reader = reader.option("aggregation.pipeline", pipeline)
```

3.3 Upsert mechanics

Figure 3.1 orients the steps below; each path through it ends in validation.

`_id`, cast to string by `sanitize_for_postgres`, is the merge key. A unique index on `_id` is ensured first; if creation fails the run falls back to plain append with a warning rather than corrupting data. Otherwise each load writes a scratch staging table (`_stg_` plus the collection name) with `mode("overwrite")`, then merges:

src/jobs/extract.py: merge_staging_into_target (excerpt)

```
WITH upsert AS (
    INSERT INTO "{POSTGRES_SCHEMA}"."{collection}" ({quoted_cols})
    SELECT {quoted_cols} FROM "{POSTGRES_SCHEMA}"."{staging_table}"
    ON CONFLICT ("{PRIMARY_KEY_COLUMN}") DO UPDATE SET {set_clause}
    RETURNING (xmax = 0) AS inserted
)
SELECT
    count(*) FILTER (WHERE inserted)     AS rows_inserted,
    count(*) FILTER (WHERE NOT inserted) AS rows_updated
FROM upsert
```

Struct, array and map columns serialize to JSON via `to_json()` (plain JDBC has no struct type); flattened names appear in the final Rich report.

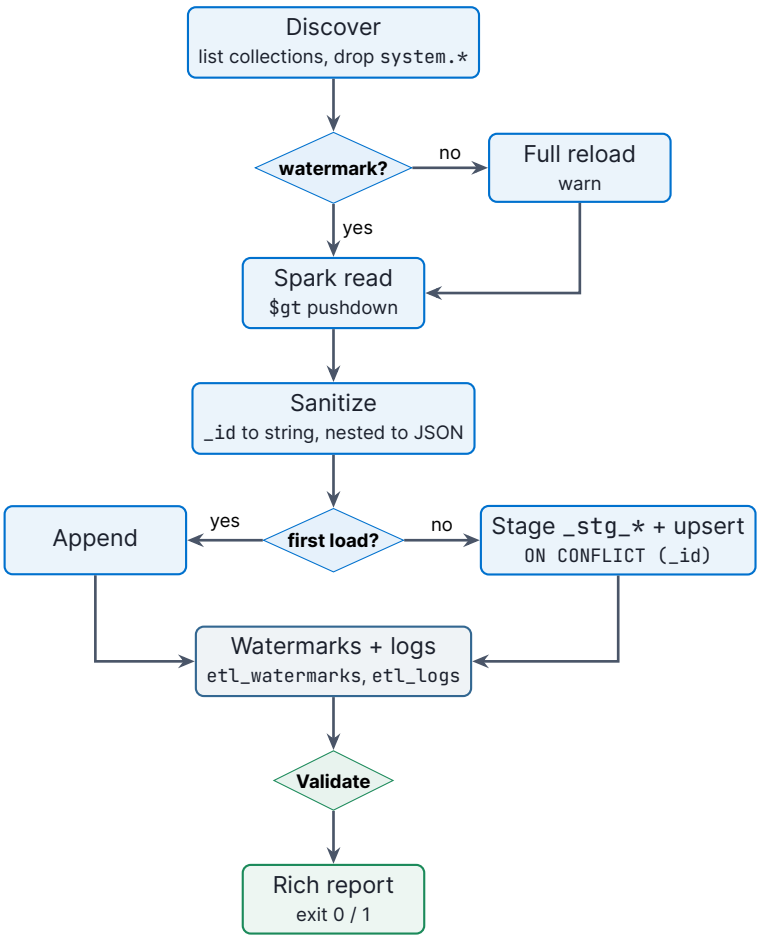


Figure 3.1: Extractor flowchart from discovery to report. Takeaway: every path ends in validation, and control tables record what happened.

3.4 Control tables and CLI

Both live beside bronze collections: `etl_watermarks` (one row per collection: incremental column, last value, run mode, counts) upserted on `table_name`, and `etl_logs` (one audit row per collection per run, best-effort). `validate_collection` recounts the target and expects `before+inserted`; any `FAILED` collection exits the process non-zero. CLI flags: `--tables`, `--full-refresh` (truncate first), `--dry-run`, `--watermark-column`.

Key takeaways

- Auto-discovery plus watermark upserts keep bronze fresh without re-copying.
- String pushdown matches the source format; the merge counts inserts versus updates exactly.
- Watermarks and logs make every run resumable and auditable.

4

Silver

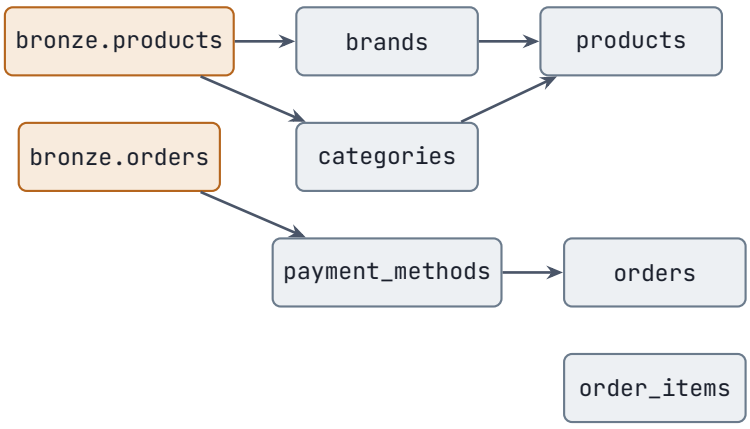
Nine models turn raw bronze into typed, deduplicated, FK-resolved tables.

At a glance

- All silver models are incremental: `merge` on natural keys, `append` for lookups.
- A 3-day lookback plus `ROW_NUMBER` dedupe absorbs late-arriving updates.
- Lookup tables (`brands`, `categories`, `payment_methods`) resolve text to surrogate keys.

4.1 Model map

Figure 4.1 shows the dependency shape; Table 4.1 names every model with its merge key.



`customers`, `employees`, `stores` and `order_items` map their bronze tables one to one

Figure 4.1: Silver dependencies. Lookup models distil bronze text columns; entity models merge and join them. Takeaway: FK resolution happens here, once.

Model	Reads	Merge key / strategy
customers	bronze customers	customer_id, merge
employees	bronze employees	employee_id, merge
stores	bronze stores	store_id, merge
orders	bronze orders + payment_methods	order_id, merge
order_items	bronze order_items	order_item_id, merge
products	bronze products + brands + categories	product_id, merge
brands	bronze products.brand	append, surrogate brand_id
categories	bronze products.category	append, surrogate category_id
payment_methods	bronze orders.payment_method	append, surrogate id

Table 4.1: The nine silver models. Source: dbt/models/silver/*.sql.

4.2 Cleaning patterns

Entity models dedupe with ROW_NUMBER()PARTITIONBY natural key ordered by updated_timestampDESC,created_timestampDESC,_idDESC, cast text to INT/NUMERIC/TIMESTAMP, normalize phones to +1-XXX-XXX-XXXX (splitting phone_extension on x), map Y to boolean, and concat names. Lookup models distil distinct normalized values with stable surrogates:

dbt/models/silver/brands.sql (excerpt)

```
{{ config(
    materialized='incremental',
    incremental_strategy='append'
) }}

WITH source_brands AS (

    SELECT DISTINCT INITCAP(TRIM(brand::VARCHAR)) AS brand_name
    FROM {{ source('bronze', 'products') }}
    WHERE
        brand IS NOT NULL
        AND TRIM(brand::VARCHAR) <> ''
```

products and orders LEFT JOIN their lookups on the normalized text, turning denormalized strings into nullable FK ids. Tests in dbt/models/silver/schema.yml pin the contracts: unique plus not-null PKs, nullable-aware relationships, non-negative ranges, an email regex (warning) and the line_amount=round(quantity*unit_price,2) expression (error).

Key takeaways

- Silver is the single place where source quirks get fixed; gold and the dashboard assume clean input.
- Incremental merge plus lookback keeps reruns cheap and late data correct.
- Nine standalone SQL checks in `tests/data-quality/silver/` re-verify the same ground outside dbt.

5

Gold star schema

Seven dimensions and one fact, Type-1, ready for BI.

At a glance

- One fact (`fact_order_items`, line-item grain) surrounded by seven dims.
- Dimensions rebuild fully each run; only the fact merges incrementally.
- `employees` deliberately has no dimension.

5.1 Schema

Figure 5.1 draws the joins; Table 5.1 states each grain. Appendix E enlarges the diagram in Fig. E.1.

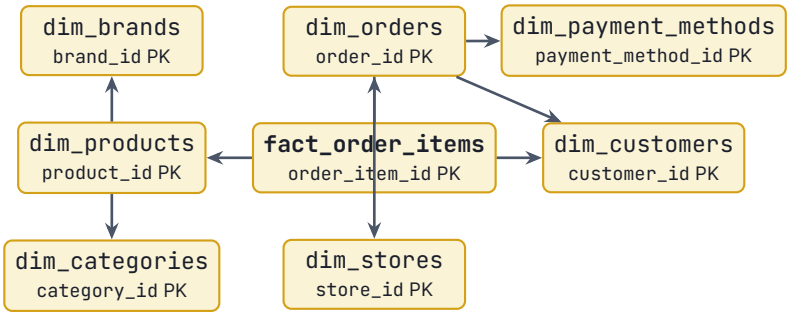


Figure 5.1: Gold ERD. Arrows are FK reads at build time. Takeaway: every analytic path joins through the fact or `dim_orders`.

Model	Grain
`dim_stores`, `dim_customers`	One row per store / customer (merge)
`dim_products`	One row per product, keeps brand/category FKs (merge)
`dim_brands`, `dim_categories`, `dim_payment_methods`	One row per lookup value (append)
`dim_orders`	One row per order header, keeps store/customer/payment FKs (merge)
`fact_order_items`	One row per order line; FKs to dims (merge on `order_item_id`)

Table 5.1: Gold models and grains. Source: `dbt/models/gold/*.sql`.

5.2 Design decisions

`fact_order_items` is the only incremental gold model, with the same 3-day lookback as silver for late lines. Dimensions full-refresh because rebuilding from silver is cheap at this scale and avoids SCD bookkeeping where history is not needed; policy is Type-1 everywhere (overwrite). Every table stamps `gold_loaded_at=CURRENT_TIMESTAMP` for lineage. The star snowflakes only at products into brands and categories, which the brand/category reports read directly.

`employees` stops at silver: headcount analytics is out of scope, so no `dim_employees` exists (also recorded in `gx/suites/gold_suites.py`). Order-header measures such as `total_amount` live on `dim_orders`; there is no `fact_orders`. Gold is gated by `dbttest--selectgold` plus the seven standalone checks in `tests/data-quality/gold/`.

Key takeaways

- Type-1 full-refresh dims plus one incremental fact is the cheapest correct shape at this scale.
- Grain statements above are the contract; row-count parity tests enforce them.
- No employees dimension is a decision, not an omission.

6

Pipeline orchestration

Eight fixed stages, five runners, one definition of success.

At a glance

- Stage order is fixed; changing it means editing `pipeline/run_pipeline.ps1` and the DAG together.
- Every runner fails fast: the first failed stage stops the run.
- Airflow adds two schedules plus an hourly health DAG.

6.1 Stages

Figure 6.1 chains the eight stages; Table 6.1 gives the exact command per stage.

#	Stage	Command
0	Preflight	`uv run python -c "import pyspark"` with a 3.5.x prefix assert
1	Extract	`uv run python src/jobs/extract.py`
2	Bronze tests	`uv run python src/jobs/sql_test.py tests/data-quality/bronze`
3	dbt silver	`uv run dbt run --select silver` then `uv run dbt test --select silver`
4	Silver tests	`uv run python src/jobs/sql_test.py tests/data-quality/silver`
5	dbt gold	`uv run dbt run --select gold` then `uv run dbt test --select gold`
6	Gold tests	`uv run python src/jobs/sql_test.py tests/data-quality/gold`
7	Great Expectations	`uv run python -m gx.run --layer all`

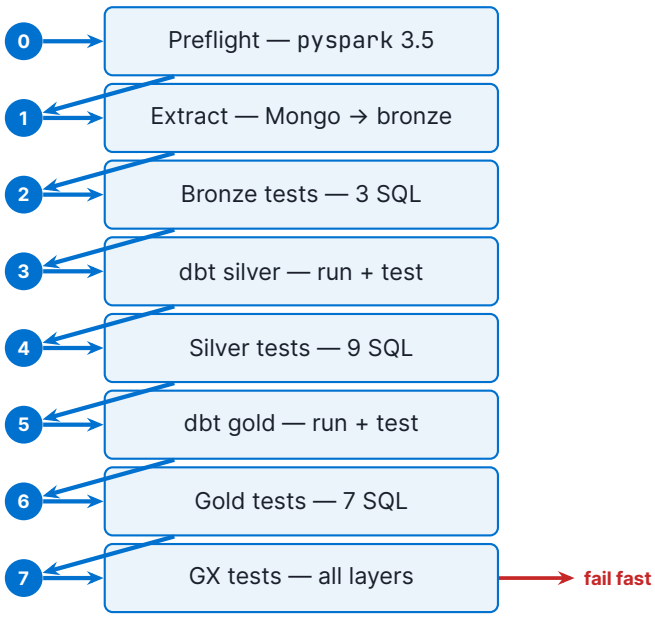


Figure 6.1: The eight stages with their tools. Banners print STEPn/7 because stages are numbered 0–7. Takeaway: order never varies, whatever runs it.

Table 6.1: Stage commands. Exit codes: `sql_test.py` 0/1/2, GX 0/1/2, extract non-zero on any failed collection.

6.2 Runner parity

Figure 6.2 states the invariant: executors differ, stages do not.

`pipeline/run_pipeline.ps1` is canonical: prerequisites check, `.env` import, per-stage logging to `logs/`, colored summary. `run_pipeline.bat` mirrors it for double-click CMD use; `main.py` mirrors it for Docker with setdefault env semantics and a container self-heal refusal; `makedata-pipeline` exposes each stage as a granular target; the Airflow DAG schedules it. Differences are environmental (env loading, working directory, logging), never logical.

6.3 Airflow

Figure 6.3 lays out the tasks; the text below gives schedules and the notification path.

`airflow/dags/walmart_pipeline_dag.py` builds two DAGs from one factory: `walmart_mwf_morning` (Monday, Wednesday, Friday 11:00 Asia/Kolkata) and `walmart_rest_night` (Sunday, Tuesday, Thursday, Saturday 23:00). Ten stage tasks run the same `uvrun...` commands after a preamble that sources `.env`, rewrites `localhost` to `host.docker.internal` and pins the venv Python. `collect_results` sweeps XCom, `choose_email`

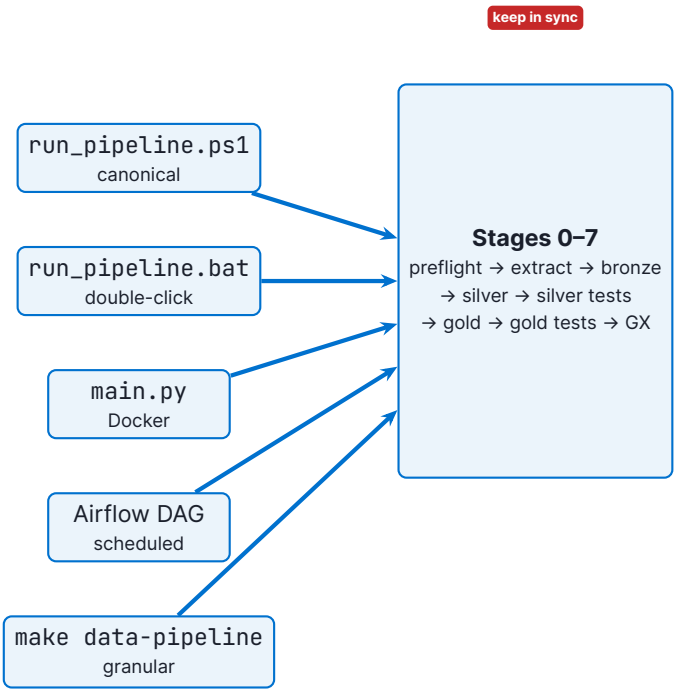
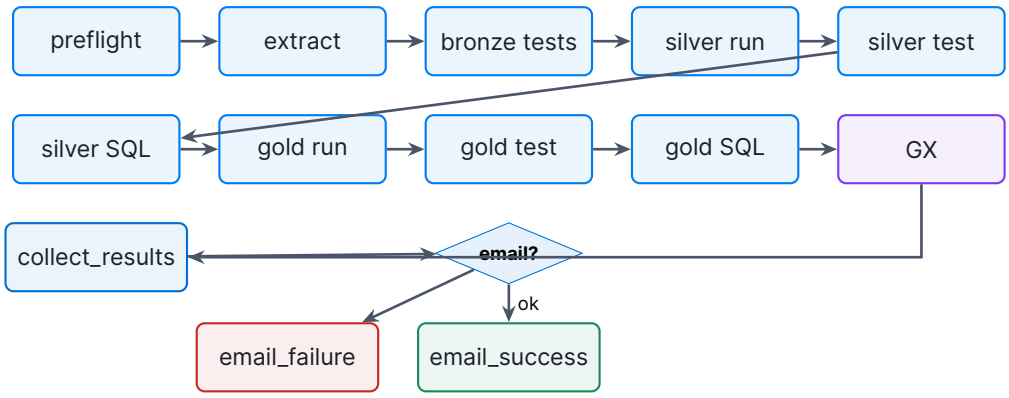


Figure 6.2: Five runners, one stage list. Takeaway: pick by platform, not by behavior.



MWF 11:00 plus rest-days 23:00 Asia/Kolkata; the health DAG runs hourly

Figure 6.3: Pipeline DAG tasks and notification branch. Takeaway: linear gate chain, then one result sweep and a single email decision.

branches, and exactly one of `email_success`/`email_failure` sends a per-stage table. `walmart_health_dag.py` pings Postgres and Mongo hourly and emails only on failure.

Key takeaways

- One stage list in five executors; parity is a maintained invariant, not an accident.
- Airflow contributes scheduling, retries and mail; the gates themselves are the same scripts.
- Fail-fast is free: PowerShell stops, Python raises, Airflow uses `all_success`.

7

Data quality

Three independent systems check the same warehouse three different ways.

At a glance

- dbt tests are blocking per-model contracts and the only declarative FK check.
- The SQL suite sweeps whole schemas and owns business aggregates plus promotion parity.
- Great Expectations runs engine-independent value checks with load-timestamp signals.

7.1 Defense in depth

Figure 7.1 maps systems to layers; the counts and unique capabilities follow.

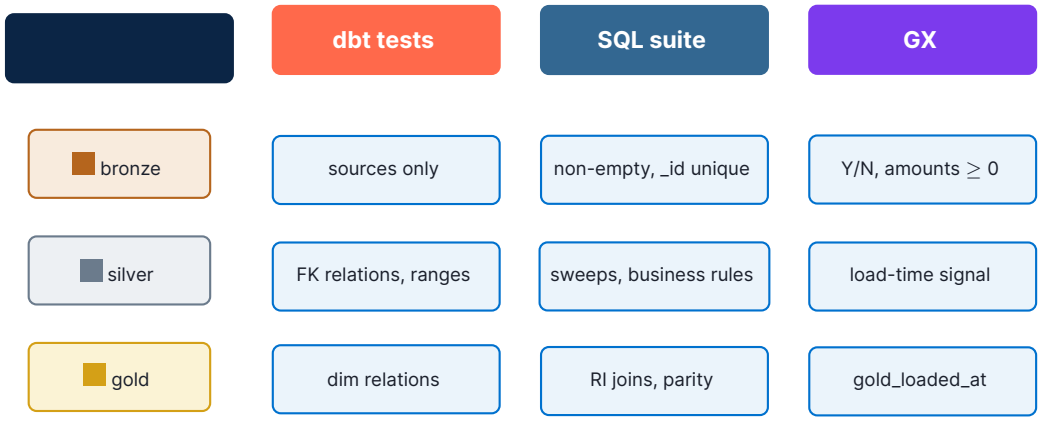


Figure 7.1: Which system checks what per layer. Overlap is deliberate redundancy across runners. Takeaway: each column catches something the others cannot.

Layer	dbt tests	SQL checks	GX validations
bronze	0 (sources declared)	3	6
silver	per-model schema	9	9
gold	per-model schema	7	8

Table 7.1: Check counts by layer and system. Sources: `dbt/models/*/schema.yml`, `tests/data-quality/`, `gx/suites/`.

7.2 What each system uniquely catches

dbt `schema.yml` tests run with each build: unique plus not-null keys, nullable-aware relationships (a `where` clause exempts legitimate null FKs), accepted ranges on amounts and the `line_amount` expression. The standalone SQL suite (`src/jobs/sql_test.py` over `tests/data-quality/`) does what dbt cannot express generically: schema-wide duplicate, null, format, negative, range and spacing sweeps, hardcoded domain allows (countries, payment methods, order statuses), six business rules including `ordertotal=SUM(lines)`, and the silver-to-gold row-count parity check, which is the only test proving promotion completeness. GX (`python-mgx.run--layerall`) validates value distributions without dbt, adds `silver_loaded_at`/`gold_loaded_at` presence as a load-job signal, and exits 1 on data failure versus 2 on build error.

Known tension, stated honestly: GX asserts FK ids `NotNull` while dbt relationships deliberately permit nulls. If unknown-brand products are legitimate, the GX suites need relaxing, not the models. Conventions: a `SELECT` returning rows fails, a `DO` block raising fails, and GX runs every requested layer unless `--fail-fast` is passed.

Key takeaways

- Three systems, three blind spots covered: declarative FKs, business aggregates with parity, runner-independent values.
- Counts in Table 7.1 are regenerated from the repo; rerun the collector after model changes.
- The GX-versus-dbt null-FK tension is open and documented, not silently resolved.

8

Data contracts

One YAML per table; humans describe, the database decides.

At a glance

- 8 bronze plus 9 silver plus 8 gold contracts cover every modeled table.
- Human fields (`description`, `transformation`, `tests`) are hand-edited; machine fields are synced.
- Drift workflow: `--check-only` gate, then sync, then `--fix-types` adoption.

8.1 Anatomy

Figure 8.1 shows where the anatomy below participates: human fields enter left, machine fields exit right.

Each file opens with identity (`apiVersion`, `kind:DataContract`, `metadata` with layer, owner, domain), then `dataset` (server, database, schema, table, grain, upstream list, dbt model, load strategy) and a `columns` list where every column carries its human contract:

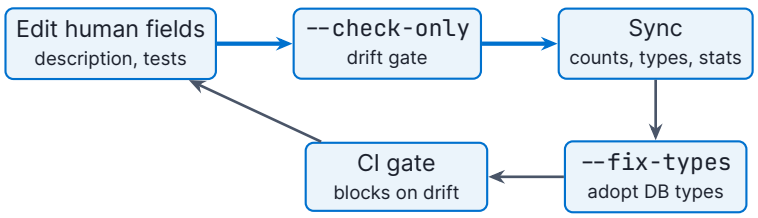


Figure 8.1: Contract lifecycle. The CI gate runs `--check-only` and blocks on drift. Takeaway: contracts stay truthful because the database rewrites them.

contracts/silver/orders.yml (excerpt, verified)

```yaml
metadata:
  name: silver.orders
  description: Deduped orders, one row per order_id. Payment_method normalized to
    FK.
dataset:
  table: orders
  grain: order_id unique, NOT NULL
  upstream:
  - bronze.orders
  - silver.payment_methods
columns:
- name: order_id
  data_type: INTEGER
  transformation: TRIM(order_id::VARCHAR)::INT
  tests:
  - not_null
  - unique
```

Machine fields sit alongside: `row_count`, `last_observed_at`, per-column `observed_data_type`, null and distinct counts, and `sync_status`. `quality.enforcedBy` points at the exact `schema.yml`, data-quality SQL and GX validation enforcing the contract. A full annotated sample is Appendix F.

8.2 Sync discipline

`uvrunpythonscripts/run_update_contract.py--check-only` reports drift and exits 1 without writing; the plain run syncs counts, types and stats; `--fix-types` adopts observed database types into the contract. `--table` and `--layer` scope the run, `--no-create` skips new files. Never hand-edit a machine field: the next sync overwrites it, and review cannot tell intent from observation afterward.

Key takeaways

- Contracts are executable documentation: tests enforce them, the sync proves them.
- The human/machine split keeps reviews meaningful and automation safe.
- Eight, nine and eight files match the modeled tables exactly; a new table needs a new contract.

9

Dashboard

Seven pages on one shared gold join graph, cached and themed.

At a glance

- Home plus six numbered pages read gold through a single `BASE_FROM` graph.
- All SQL lives in `helpers/queries.py`; pages stay thin by rule.
- Walmart theme, period-over-period KPIs and a data-trust banner.

9.1 Layering

Figure 9.1 traces a request; the rules below keep the layers honest.

Pages import only from `dashboard.lib.queries` (a re-export shim); figure builders live in `lib/charts.py`; shared widgets in `helpers/components.py`. `Filters` is a frozen dataclass so it hashes as a `st.cache_data` key: filter options cache 3600 s, data queries 60–300 s. Every query aggregates in Postgres and filters soft-deletes; the only bounded raw pull is the 200-row recent-orders sample. `end_date` is an exclusive upper bound, empty selections match nothing and render `empty_state`, and all HTML passes through `_safe_html`.

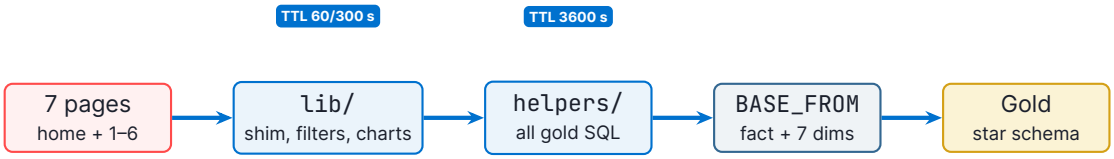


Figure 9.1: Dashboard data path with cache TTLs. Takeaway: one join graph, one SQL file, one theme.

9.2 Pages

Table 9.1 inventories the seven pages; each row follows the layering above.

Page	Content
Home	KPI cards with prior-period deltas, sparklines, freshness badge
1_Sales	Revenue trend, store and payment mix, day-hour heatmap
2_Products	Top products, brand bars, treemap, pareto, bubble
3_Customers	RFM scatter, cohort heatmap, top customers, order search
4_Categories	Sunburst, status funnel, share area
5_Geo	Store map with click-to-filter (synthetic coordinates) plus province bars
6_Data_Health	Table counts, pipeline status from bronze control tables

Table 9.1: Dashboard pages. Source: `dashboard/pages/*.py`.

Geo coordinates are synthetic state-capital points, not real addresses, and are labelled as such. Deployment reads `st.secrets` through `lib/db.py` on Streamlit Cloud and `.env` locally; `requirements.txt` stays minimal and databricks-free, guarded by the CI dashboard-smoke job. Entry point is `dashboard/home.py`; any `app.py` reference is stale.

Key takeaways

- One join graph means every page agrees on what revenue means.
- Thin pages plus cached aggregates keep interactions fast without a semantic layer.
- Synthetic geo and secrets discipline are stated in the open, not discovered later.

10

CI, CD and Docker

Seven CI jobs gate merges; two images ship; releases cut on tags.

At a glance

- `ci.yml` runs lint, three test jobs, DAG integrity, Docker build and a live-Postgres integration.
- CD publishes rolling tags on green main; release publishes semver on tags; refresh is manual until Neon is ready.
- Two images (pipeline runner, Airflow stack) plus a local Compose file.

10.1 CI

Figure 10.1 graphs the jobs; Table 10.1 states each gate.

Job	Gate
lint	Ruff check plus format, SQLFluff on `dbt/modelssql/tests/`
dashboard-smoke	Dashboard-only install, assert no databricks, import check
unit-tests	`pytesttests/unit` (30 tests, mocked)
smoke-tests	Imports, layout, CLIs, DAG structure (16 tests)
dag-integrity	Constraints-pinned Airflow, DagBag import plus cycle check
docker-build	Both Dockerfiles, no push
integration	Seed bronze from fixtures, then bronze checks, dbt silver, silver checks, dbt gold, gold checks, GX all

Table 10.1: CI jobs. Source: `.github/workflows/ci.yml`.

`dbt.yml` gives cheap parse-only feedback on `dbt/**` changes. `cd.yml` pushes rolling plus SHA tags after green main; `release.yml` validates semver tags and publishes both images with a GitHub Release; `refresh.yml` rebuilds Neon gold on demand (cron commented out until secrets land). Pre-commit mirrors lint locally with the Conventional Commits hook.

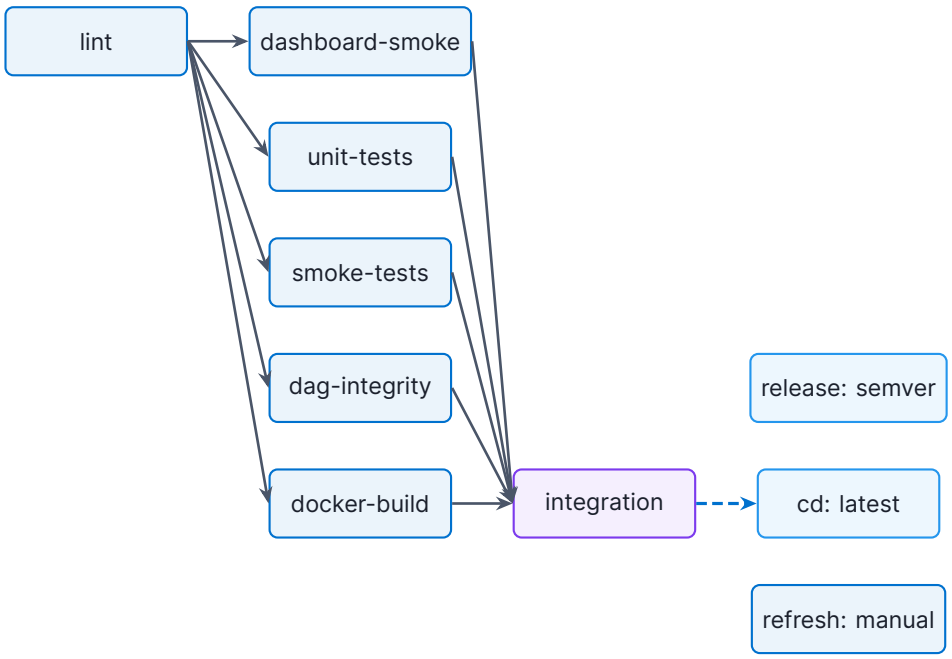


Figure 10.1: CI job DAG and publish triggers. Takeaway: everything expensive waits on lint; integration replays the pipeline order against live Postgres.

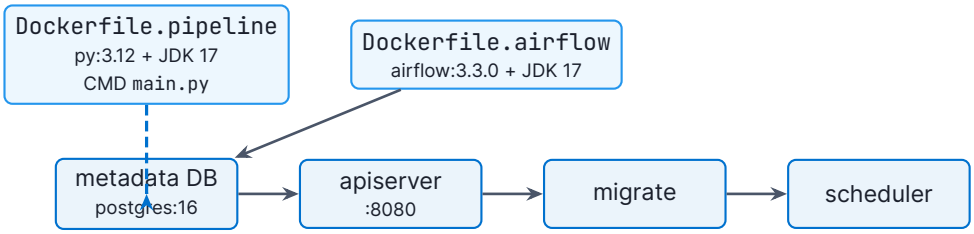


Figure 10.2: Images and Compose services. Takeaway: one image runs the pipeline, the other schedules it, Compose wires them locally.

10.2 Docker

Figure 10.2 places the images beside the stack they serve.

`Dockerfile.pipeline` (Python 3.12-slim plus JDK 17, `uv sync --frozen`, `dbt deps`, entrypoint script, default `uv run main.py`) is the standalone runner published to GHCR. `Dockerfile.airflow` (Airflow 3.3.0 plus JDK 17, repo mounted, no code copy) backs `compose.yml`: metadata Postgres, API server on 8080, migrate and scheduler on LocalExecutor. The entrypoint requires Postgres vars, rewrites localhost to `host.docker.internal` and TCP-waits for the database before exec.

Key takeaways

- CI replays production order, so a green main means the pipeline logic is proven, not just linted.

- Publish paths do not overlap: rolling tags from merges, semver from tags, Neon refresh by hand.
- JDK 17 is pinned deliberately; the Mongo connector is untested on JDK 21.

11

Operations and security

Health checks, secret scanning, backups and least privilege.

At a glance

- Health and security scripts are read-only and never print secrets.
- The public dashboard connects as least-privilege `dashboard_ro`.
- Demo data is PII-scrubbed; logs rotate before they grow.

11.1 Operate

`scripts/run_health_check.py` probes Postgres, Mongo, PySpark, Docker, Compose, the Airflow API, the newest pipeline log and git state, then renders `HEALTHY`, `HEALTHYWITHWARNINGS` or `ACTIONNEEDED`; only `FAIL` exits non-zero, with `--quick` skipping Spark. `scripts/monitor_logs.sh` summarizes or prunes `logs/` (newest always kept), `scripts/backup.sh` dumps and restores per schema with `pg_dump`/`pg_restore`, and `scripts/db_shell.sh` opens psql with the right search path. Every logger writes console plus a daily file through rotating 5 MB × 5 handlers, so a failed collection can be diagnosed three ways (console, file, `etl_logs`) without rerunning.

11.2 Secure

`scripts/run_security_check.py` scans tracked text files for private keys, cloud tokens, password-bearing URIs and hardcoded secrets, reporting paths and lines but never values, then checks env hygiene, Airflow defaults, Dockerfile users, lockfiles and CI scanning. `sql/security/dashboard_readonly_role.sql` creates `dashboard_ro` with usage on gold plus select on present and future tables, so the public app never holds owner credentials. `scripts/seed_demo_db.py` copies gold to a separate demo database with names, emails

and phones anonymized. `.env` and `secrets.toml` are git-ignored; only dummy-value examples are committed, guarded by the private-key pre-commit hook.

Key takeaways

- Read-only auditing plus least-privilege access beats post-incident cleanup.
- Three independent log sinks mean failures are explainable after the fact.
- No credential ever appears in code, docs, logs or this report.

12

Results and metrics

Every chart below is generated from `reports/data/*.csv`; nothing is hand-drawn from memory.

At a glance

- Component, test and row-count CSVs come from `uvrunpythonscripts/report/collect_metrics.py`.
- Durations come from the newest `logs/pipeline_*.log`; unknown cells stay blank.
- Bronze counts of one row per table are the seeded snapshot recorded in the contracts, not production volumes.

12.1 Components and coverage by layer

Figure 12.1 counts the machinery; Fig. 12.2 shows how the checks compose.

12.2 Row counts

Figure 12.3 proves promotion parity on a log scale; Fig. 12.4 records the seeded bronze snapshot for context.

12.3 Code size and test composition

Figure 12.5 sizes the areas; Fig. 12.6 shapes the suite and Fig. 12.7 numbers it.

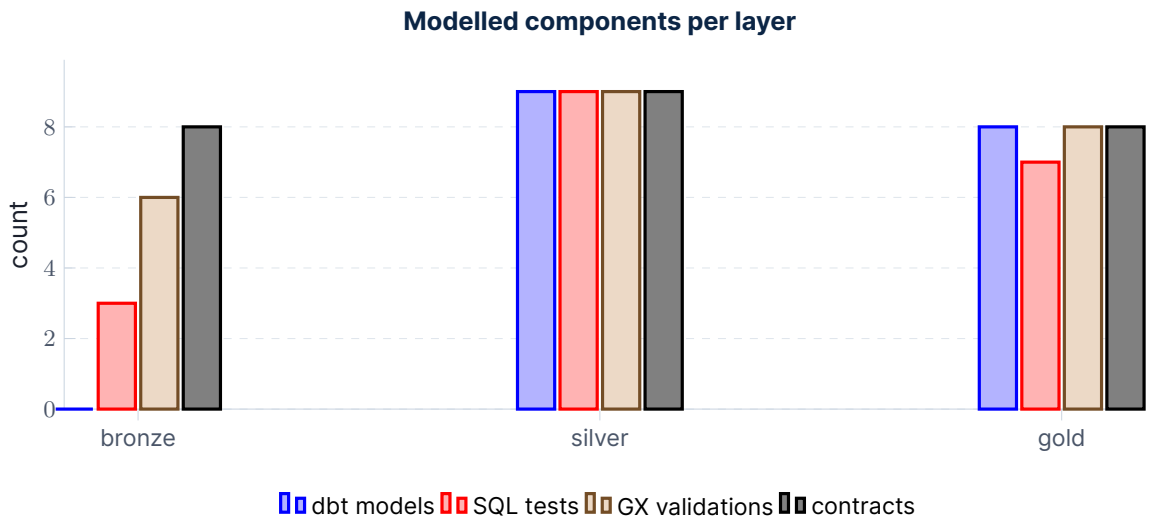


Figure 12.1: Components per layer: bronze has no dbt models (sources are declared, not built). Takeaway: silver carries the most machinery. Source: `reports/data/components_by_layer.csv`.

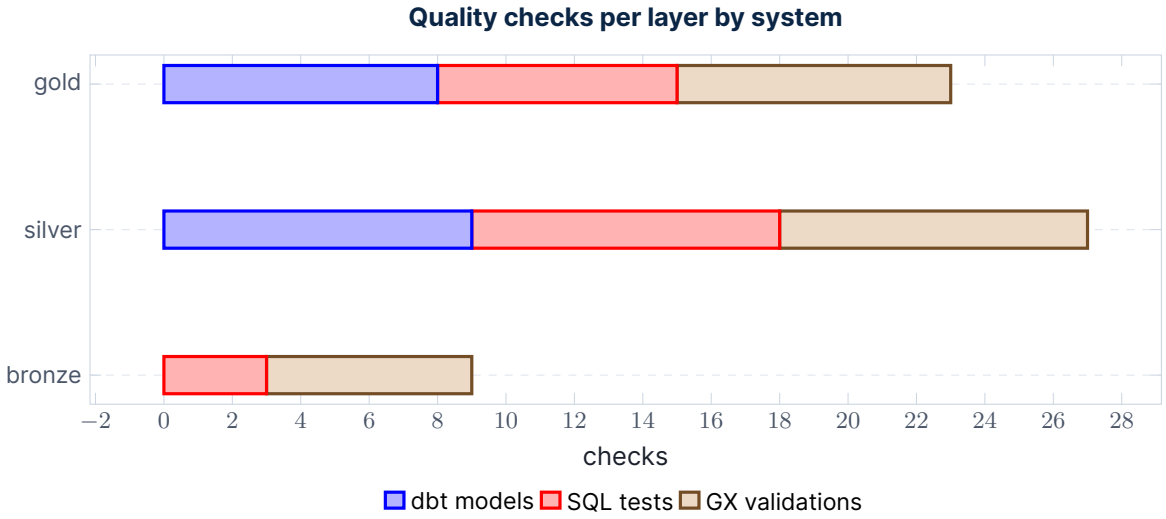


Figure 12.2: Defense-in-depth composition. Takeaway: no layer relies on a single system. Source: `reports/data/components_by_layer.csv`.

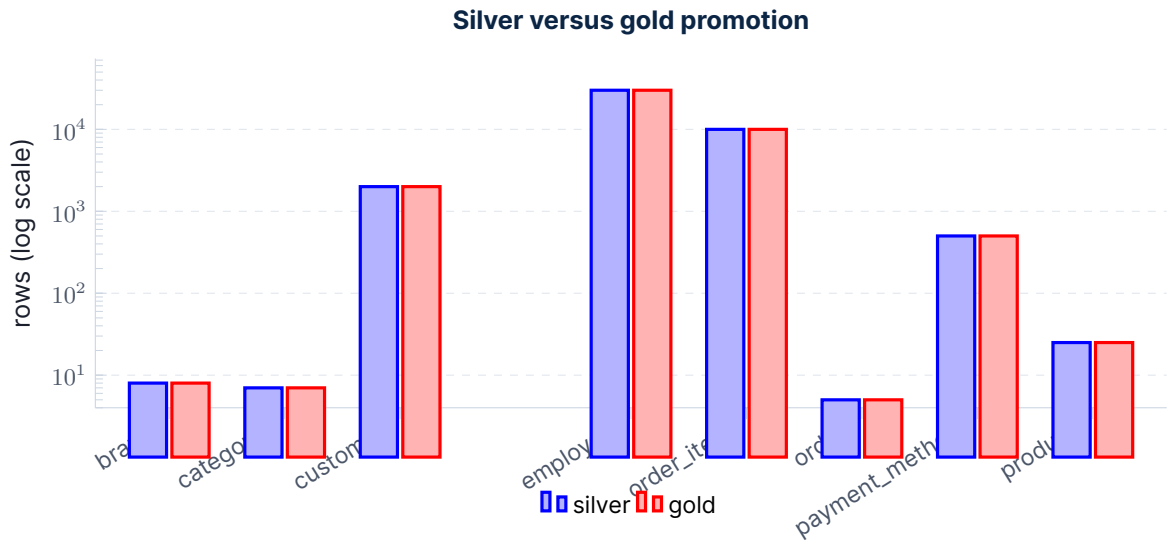


Figure 12.3: Promotion parity on a log axis; `employees` is skipped because it has no gold dimension by design. Takeaway: every promoted table arrives complete. Source: `reports/data/layer_rowcounts.csv` (contract machine fields).

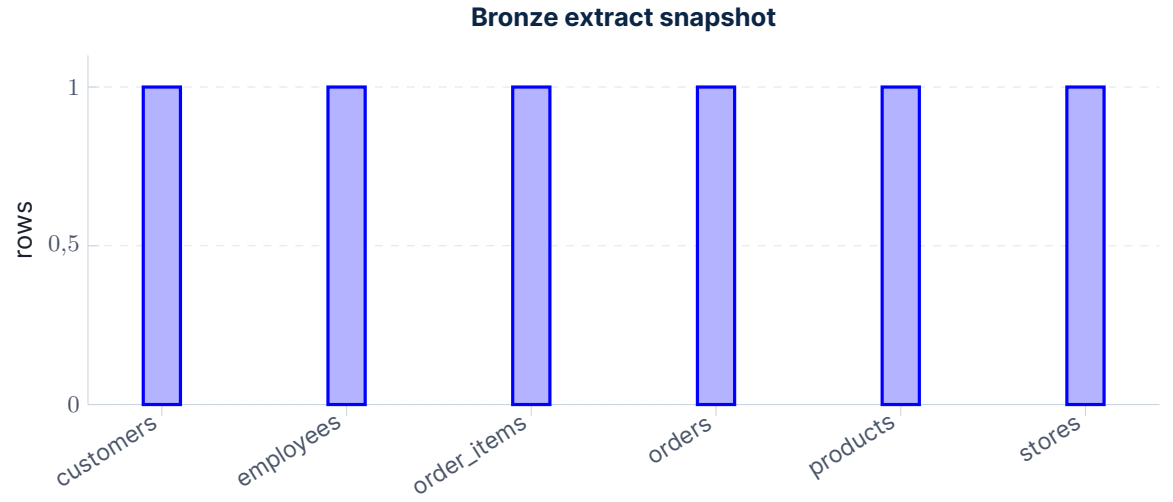


Figure 12.4: Bronze counts are one row per table: the seeded snapshot recorded in the contracts, shown to prove the pipeline, not production volume. Source: `reports/data/extract_rows.csv`.

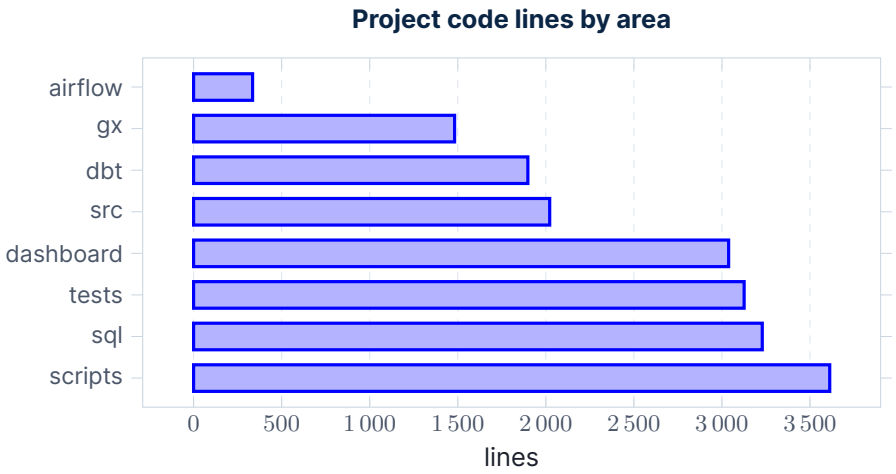


Figure 12.5: Python, SQL, YAML and shell lines per area, excluding vendored and generated trees. Takeaway: ops scripts outweigh the runtime; the warehouse itself is compact. Source: `reports/data/loc_by_area.csv`.

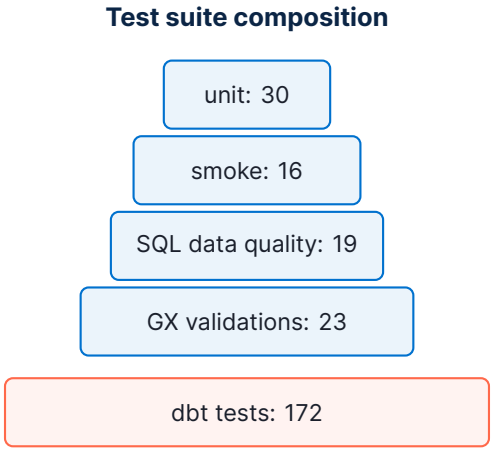


Figure 12.6: Suite sizes as test-function and check counts. Takeaway: dbt carries two thirds of all checks. Source: `reports/data/tests_pyramid.csv`.

12.4 Stage durations

Figure 12.8 times the latest complete run from its log.

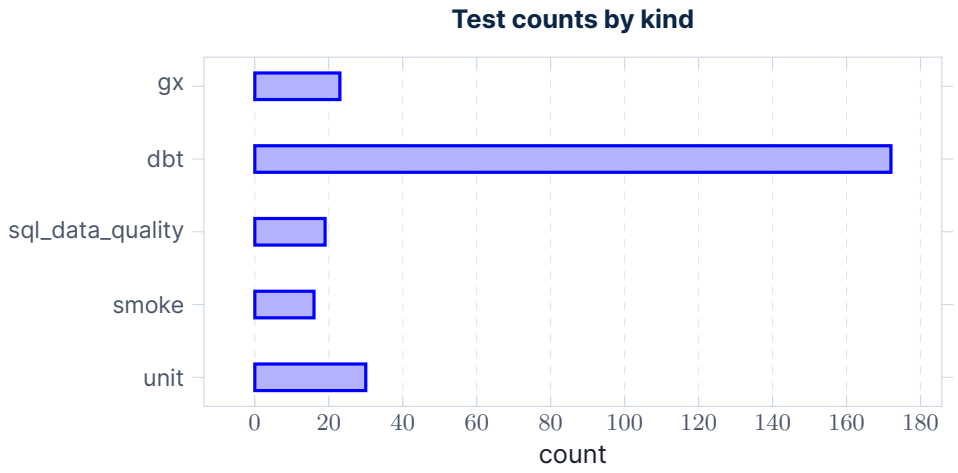


Figure 12.7: Exact counts behind Fig. 12.6: unit and smoke count defined test functions; dbt counts `tests:` entries in `schema.yml`. Source: `reports/data/tests_pyramid.csv`.

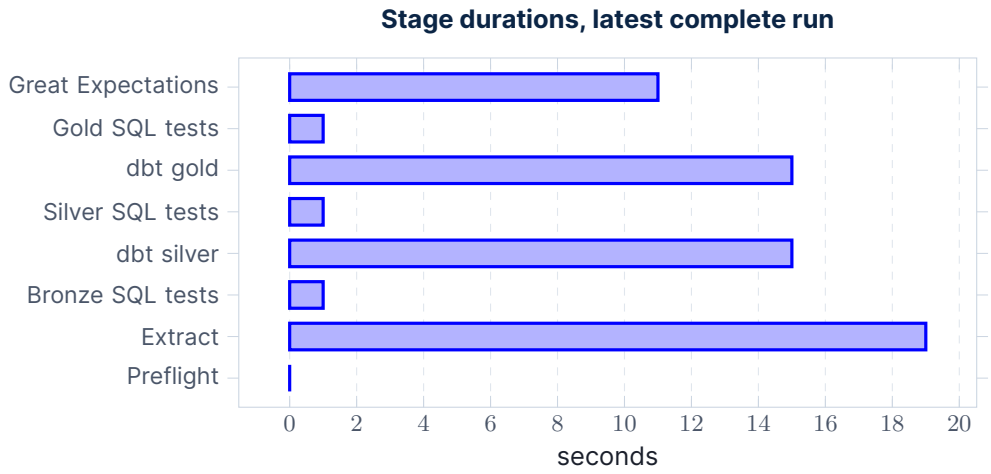


Figure 12.8: Extract and the dbt builds dominate; SQL gates take about a second. Source: newest `logs/pipeline_*.log` via the collector.

Page	Charts	Gold tables
1_Sales	8	dim_brands; dim_categories; dim_customers; dim_orders; dim_payment_methods; dim_products; dim_stores; fact_order_items
2_Products	8	dim_brands; dim_categories; dim_customers; dim_orders; dim_payment_methods; dim_products; dim_stores; fact_order_items
3_Customers	4	dim_brands; dim_categories; dim_customers; dim_orders; dim_payment_methods; dim_products; dim_stores; fact_order_items
4_Categories	9	dim_brands; dim_categories; dim_customers; dim_orders; dim_payment_methods; dim_products; dim_stores; fact_order_items
5_Geo	3	dim_brands; dim_categories; dim_customers; dim_orders; dim_payment_methods; dim_products; dim_stores; fact_order_items
6_Data_Health	0	dim_brands; dim_categories; dim_customers; dim_orders; dim_payment_methods; dim_products; dim_stores; fact_order_items

Table 12.1: Pages, figure-builder calls and gold tables read. Every business page shares the `BASE_FROM` graph; the health page additionally reads bronze control tables. Source: `reports/data/dashboard_pages.csv`.

12.5 Dashboard coverage

Table 12.1 reads directly from the collector output with no hand edits.

Key takeaways

- Eight charts plus the coverage table, all regenerated by one command.
- Log-scale promotion chart, honest seeded bronze counts, dated durations.
- Rerun `makereport-data` after any model, test or page change before rebuilding.

13

Limitations and roadmap

What the system does not do yet, stated plainly, then what comes next.

At a glance

- Batch only, Type-1 only, synthetic demo geography.
- Two honest inconsistencies are documented, not hidden.
- The refresh cron and Data Docs publishing wait on operations, not code.

13.1 Limitations

Table 13.1 names each limit; the paragraphs below own up to the inconsistencies.

Area	Limitation
Freshness	Batch incremental only; no streaming or CDC.
History	Type-1 everywhere; silver keeps no SCD2 snapshots.
Geography	Store coordinates are synthetic state-capital points.
Employees	Modelled to silver only; no headcount analytics.
Refresh	Neon gold refresh cron is commented out until secrets land.
Docs	GX Data Docs are not published from checkpoints.
PII	No column masking; demo seeding scrubs instead.

Table 13.1: Known limits. Each has a roadmap item below.

Two inconsistencies deserve honesty. First, some docs say `docker/Dockerfile` while the files are `Dockerfile.pipeline` and `Dockerfile.airflow`; the files win. Second, banners print `STEPn/7` for eight stages because numbering starts at zero; the count is stages minus one, not an off-by-one in the runner. Third, GX asserts non-null FK ids while dbt relationships

permit nulls (see Chapter 7); until product decides whether unknown lookups are legitimate, both stay as they are.

13.2 Roadmap

Near term: enable the Neon refresh cron, publish Data Docs, resolve the null-FK question, add bronze `schema.yml` depth. Medium term: dbt snapshots for customer and store history, BSON-date migration for native watermark pushdown, column masking for external gold access. Long term: change-stream CDC for near-real-time bronze.

Key takeaways

- Limits are scoped and each maps to a concrete next step.
- Inconsistencies are documented where users will trip over them.
- Nothing on the roadmap requires re-architecture, only iteration.

A

File inventory

The tracked tree grouped by area; generated from `gitls-files`.

Path	Purpose	Area
`main.py`	Docker pipeline entrypoint	pipeline
`run_pipeline.bat`	Root CMD runner	pipeline
`pipeline/run_pipeline.ps1`	Canonical PS runner	pipeline
`Makefile`	Portable task runner	build
`pyproject.toml, uv.lock`	Deps source of truth	build
`src/utils/engine.py`	Validated config	runtime
`src/utils/connection.py`	Cached factories	runtime
`src/utils/logger.py`	Console+file logging	runtime
`src/jobs/extract.py`	Mongo to bronze	runtime
`src/jobs/sql_test.py`	SQL gate runner	quality
`src/jobs/run_dbt_models.py`	dbt helper	runtime
`src/jobs/ci_seed_bronze.py`	CI fixture loader	quality
`scripts/run_*.py`	Ops wrappers (7)	ops
`scripts/*.sh`	Shell helpers (7)	ops
`scripts/report/*.py`	Report collectors	report
`dbt/dbt_project.yml, profiles.yml`	Project + targets	dbt
`dbt/models/bronze/source.yml`	Source declarations	dbt
`dbt/models/silver/*.sql (9)`	Clean models	dbt
`dbt/models/silver/schema.yml`	Silver tests	quality
`dbt/models/gold/*.sql (8)`	Dims + fact	dbt
`dbt/models/gold/schema.yml`	Gold tests	quality
`dbt/tests/generic/*.sql (3)`	Reusable tests	quality
`contracts/*/*.yml (25)`	Table contracts	quality
`gx/run.py, context.py`	GX runner + context	quality
`gx/suites/*.py (3)`	Layer validations	quality
`tests/data-quality/*/*.sql (19)`	SQL checks	quality
`tests/unit/*.py (6)`	Unit tests	quality

Path	Purpose	Area
tests/smoke/*.py (5)	Smoke tests	quality
tests/fixtures/bronze/*.json (6)	Offline fixtures	quality
dashboard/home.py, pages/ (7)	Streamlit app	app
dashboard/lib/, helpers/	Queries, charts, theme	app
dashboard/requirements.txt	Cloud deps	app
airflow/dags/*.py (2)	ETL + health DAGs	orchestration
docker/Dockerfile.* (2)	Images	ops
docker/compose.yml	Local stack	ops
sql/analytics/*.sql (16)	BI queries	app
sql/functions, triggers, procedure	Server logic	app
sql/security/*.sql	Readonly role	ops
jars/*.jar (6)	Spark connectors	runtime
docs/*.md (14)	Deep dives	docs
reports/main.tex, latex/, chapters/	This report	report
reports/figures/tikz/*.tex (14)	Diagrams	report
reports/data/*.csv (8)	Chart data	report
.github/workflows/*.yml (8)	CI/CD	ops

B

Command cheat-sheet

The commands that run this repo, verified against `AGENTS.md` and the Makefile.

Table B.1 is the whole appendix: one table, no prose needed.

Task	Command
Setup	`uvvenv\&\&uvsync`
Full pipeline	`makedata-pipeline`
Windows CMD	`run_pipeline.bat`
PowerShell	`.\pipeline\run_pipeline.ps1`
Docker	`uvrunmain.py`
Static checks	`makestatic-checks`
Unit tests	`makeunit-test`
All tests	`uvrunpytest`
SQL suite	`uvrunpythonsrc/jobs/sql_test.pytests/data-quality/silver`
GX	`uvrunpython-mgx.run--layerall`
Contracts drift	`uvrunpythonscripts/run_update_contract.py--check-only`
Dashboard	`uvrunstreamlitrundashboard/home.py`
Report	`makereport`

Table B.1: Cheat-sheet.

C

Environment variable reference

Names only, from `.env.example`; values live in local files and secrets.

Variable	Purpose
`POSTGRES_HOST, POSTGRES_PORT`	Warehouse host and port
`POSTGRES_DATABASE`	Database name (`walmart_db`)
`POSTGRES_USERNAME, POSTGRES_PASSWORD`	Owner credentials (never committed)
`POSTGRES_SSLMODE, POSTGRES_CHANNEL_BINDING`	TLS for managed providers
`POSTGRES_SCHEMA_BRONZE/SILVER/GOLD`	Layer schema names
`MONGO_URI, MONGO_DB`	Source connection and database
`PYSPARK_PYTHON, PYSPARK_DRIVER_PYTHON`	Spark worker interpreter
`DATABRICKS_HOST/PATH/TOKEN`	Databricks SQL warehouse (optional)
`DATABRICKS_CATALOG/SCHEMA`	Sync target namespace

D

Glossary

Acronyms and domain terms as used in this report.

Table D.1 defines every term the chapters assume.

Term	Meaning
Medallion	Bronze/raw, silver/clean, gold/modelled lakehouse pattern
SCD / Type-1	Slowly changing dimension; Type-1 overwrites history
Grain	Business meaning of one row (e.g. one order line)
Dedup	Removing duplicate source rows, here by `ROW_NUMBER`
Watermark	High-water mark (timestamp) bounding incremental reads
Upsert	Insert-or-update merge keyed on `_id`
DAG	Directed acyclic graph; Airflow workflow definition
XCom	Airflow cross-task message passing
dbt	Data build tool; SQL models with tests and lineage
GX	Great Expectations, data validation framework
Neon	Managed Postgres hosting the production backend
GHCR	GitHub Container Registry, image host
PII	Personally identifiable information
TTL	Cache time-to-live for dashboard queries

Table D.1: Glossary.

E

Full entity-relationship diagram

Every gold key and relationship on one landscape page.

Figure E.1: Gold ERD, repeated large for reference. The fact joins dims on surrogate and natural keys; products snowflake into brands and categories. Source: `dbt/models/gold/*.sql`.

F

Sample data contract

`contracts/silver/orders.yml`: human fields editable, machine fields synced.

contracts/silver/orders.yml, lines 1–45 (verified)

```yaml
apiVersion: 1.0.0
kind: DataContract
metadata:
  name: silver.orders
  layer: silver
  version: 1.0.0
  status: active
  owner: analytics-engineering
  domain: retail-warehouse
  description: Deduped orders, one row per order_id. Payment_method normalized to
    FK.
dataset:
  server: postgres-prod
  database: walmart_db
  schema: silver
  table: orders
  grain: order_id unique, NOT NULL
  upstream:
  - bronze.orders
  - silver.stores
  - silver.customers
  - silver.payment_methods
  dbtModel: dbt/models/silver/orders.sql
  loadStrategy: incremental merge on order_id with 3-day lookback
  row_count: 10000
  column_count_contract: 11
  column_count_observed: 11
  table_exists: true
  last_observed_at: '2026-10-13T14:51:42+00:00'
columns:
- name: order_id
  data_type: INTEGER
  nullable: true
```

```
  unique: true
  primary_key: true
  transformation: TRIM(order_id::VARCHAR)::INT
  tests:
  - not_null
  - unique
  observed_data_type: INTEGER
```

Editable: `description`, `transformation`, `tests`. Synced: `row_count`, `last_observed_at`, `observed_*` and counts. The full file runs 208 lines; the remaining columns follow the same shape.

G

Change log

What changed in the report itself, newest first.

Table G.1 is short today; it grows with the report.

Date	Change
2026-10-14	Rebuilt report in the new design system (v0.2.2, `54c9cae`)
2026-09	Previous `article`-class report (preserved in `reports/legacy/`)

Table G.1: Report change log.